

Overview on CVE and Tools Used

Version 1.0.0, 2026-09-22

Home

Introduction

Course Title: Overview on CVE and Tools Used

Description: Course description FIXME

Duration: FIXME hours

Topics covered in this course

- Understanding CVE Fundamentals
- CVE Resolution Workflow
- Triage and Tracking Tools
- Impact Analysis and Verification
- Build and Release Infrastructure

Prerequisites

This course assumes that you have the following prior experience:

- Course or experience...
- Course or experience...
- Course or experience...



Use of Artificial Intelligence: This publication contains content or media that is generated or assisted by AI tools. All the Training deliverables have primarily human authorship, and all content was reviewed by a human domain expert.

Lab Environment

FIXME: Instructions for accessing the lab environment for this hands-on based training.

Understanding CVE Fundamentals

Understanding CVE Fundamentals

Common Vulnerabilities and Exposures (CVE) provides a standardized, unique identifier for publicly known cybersecurity vulnerabilities. In RHEL Sustaining Engineering, understanding these fundamentals is critical for maintaining software integrity and security across supported release lifecycles.

Key aspects include:

- **Vulnerability Triage and Scoring:** Product Security (ProdSec) assesses vulnerabilities using the Common Vulnerability Scoring System (CVSS) to determine severity levels—Low, Moderate, Important, or Critical.
- **Sustaining Engineering Scope:** Engineering efforts focus on applying security fixes across extended life streams (EUS, TUS, E4S, AMC, ELS) to ensure long-term stability for enterprise environments.
- **Backporting Strategy:** To maintain API/ABI compatibility and prevent regressions, engineers backport minimal, targeted security patches to specific release streams rather than performing full upstream version rebases.

FIXME: [Insert block diagram illustrating the relationship between CVE severity, CVSS scoring, and the RHEL maintenance lifecycle]

Definition and purpose of Common Vulnerabilities and Exposures (CVE)

Overview on CVE and Tools Used

Understanding CVE Fundamentals

Definition and Purpose of Common Vulnerabilities and Exposures (CVE)

In the landscape of enterprise software security, maintaining the integrity of operating systems like Red Hat Enterprise Linux (RHEL) requires a systematic approach to identifying, tracking, and remediating security flaws. The cornerstone of this ecosystem is the **Common Vulnerabilities and Exposures (CVE)** system.

What is a CVE?

A CVE is a standardized, unique identifier assigned to a publicly known information-security vulnerability. The CVE program, sponsored by the Cybersecurity and Infrastructure Security Agency (CISA), provides a common language for security professionals, vendors, and researchers to communicate about vulnerabilities.

Each CVE entry includes: * **Unique ID:** A permanent, globally recognized identifier (e.g., CVE-2023-

XXXXX). * **Description:** A brief, technical explanation of the security flaw. * **References:** Links to patches, technical reports, and advisory information.

Purpose of the CVE System

The primary purpose of the CVE system is to facilitate the efficient management of security risks. Within the RHEL Sustaining Engineering lifecycle, CVEs serve three critical functions:

1. **Standardization:** By providing a universal identifier, CVEs prevent ambiguity. When a researcher reports a vulnerability in a Linux kernel subsystem, the CVE identifier ensures that Product Security (ProdSec) and RHEL Subsystem Teams (SSTs) are referencing the exact same issue across different tracking platforms.
2. **Prioritization:** CVEs are accompanied by **Common Vulnerability Scoring System (CVSS)** scores. These scores provide a quantitative measure of the vulnerability's severity (Low, Moderate, Important, Critical), which dictates the urgency of the response and the resources required for remediation.
3. **Traceability:** The CVE ID allows stakeholders to track a vulnerability from initial discovery and triage through the development of a patch, all the way to the final delivery of an errata update to the customer.

FIXME: [Flowchart: The CVE Lifecycle—from public disclosure to customer update, illustrating the flow from CVE ID assignment to Red Hat Security Advisory (RHSA) release.]

The Role of CVEs in RHEL Sustaining Engineering

While standard product engineering focuses on active, in-development releases, RHEL Sustaining Engineering is responsible for security maintenance across Extended Life Streams (EUS, TUS, E4S, AMC, and ELS). In this context, CVEs act as the primary triggers for maintenance workflows.

When a CVE is assigned, the RHEL process determines the remediation path: * **Triage:** ProdSec evaluates the CVE to determine its relevance and impact on specific RHEL versions. * **Backporting Strategy:** To maintain binary (ABI) and application programming interface (API) compatibility, engineers rarely rebase entire upstream versions. Instead, the CVE data helps developers isolate and "backport" only the necessary security fixes into the specific, stable release stream.

This methodology ensures that customers receive critical security updates without the risk of system-wide regressions that might occur during a full version upgrade.

Vulnerability triage and CVSS severity scoring

Vulnerability Triage and CVSS Severity Scoring

In the lifecycle of RHEL security maintenance, **Vulnerability Triage** is the foundational step that dictates the urgency, resources, and technical strategy applied to a security flaw. When a Common Vulnerabilities and Exposures (CVE) identifier is assigned, it serves as the key for tracking, but the CVSS (Common Vulnerability Scoring System) score provides the actionable intelligence required for prioritization.

The Triage Process

Triage is the collaborative process where Product Security (ProdSec) and Sustaining Engineering teams evaluate the relevance of a CVE to the RHEL ecosystem. During this phase, the vulnerability is assessed to determine:

- **Applicability:** Does the vulnerable code exist in the RHEL version, and is it compiled or linked in a way that makes it reachable?
- **Context:** Does the specific configuration or environment of RHEL mitigate the impact?
- **Prioritization:** Where does this issue sit in the current maintenance queue based on its severity?

FIXME: [Block Diagram: The CVE Triage Workflow showing inputs from ProdSec, architectural analysis, and output into Jira/Bugzilla trackers]

Understanding CVSS Severity Scoring

CVSS provides a standardized framework for assessing the severity of a vulnerability. In Red Hat's ecosystem, these scores are converted into a qualitative severity rating, which directly influences the Service Level Agreements (SLAs) and the release vehicle (e.g., Z-stream vs. Y-stream) for the fix.

The scoring is based on three main metric groups:

1. **Base Metrics:** Represent the intrinsic qualities of a vulnerability that are constant over time and across user environments. This includes exploitability (Attack Vector, Complexity) and impact (Confidentiality, Integrity, Availability).
2. **Temporal Metrics:** Reflect the characteristics of a vulnerability that change over time, such as the availability of exploit code or the release of a remediation patch.
3. **Environmental Metrics:** Represent the characteristics of a vulnerability that are unique to a user's environment.

Severity Ratings Table

Red Hat maps the CVSS numerical score (0.0 to 10.0) to the following severity levels:

Severity	CVSS Range	Definition/Impact	:---	:---	:---	Critical	9.0 – 10.0	Exploitable remotely with no user interaction; results in full system compromise.
						Important	7.0 – 8.9	Exploitable but may require specific configuration or user interaction; impacts confidentiality/integrity.
						Moderate	4.0 – 6.9	Difficult to exploit or limited impact; requires local access or specific conditions.
						Low	0.1 – 3.9	Minimal impact; often requires specialized conditions to exploit.

Impact on Sustaining Engineering

The triage and severity assignment dictate the "Sustaining Engineering" path:

- **Critical/Important:** These vulnerabilities almost always mandate immediate action. They trigger full security advisory workflows, requiring backports into existing active RHEL streams (EUS, TUS, E4S, etc.).

- **Moderate/Low:** These are prioritized based on resource availability and release timelines. They typically do not trigger emergency Z-stream releases but are instead addressed in the next scheduled Y-stream or maintenance update.

Hands-on Activity: Analyzing a CVE Tracker

In this activity, you will navigate the internal tracking system to understand how severity drives the engineering workflow.

Scenario: You have been assigned a Jira ticket for a new CVE.

1. **Access the Tracker:** Open the Jira/Bugzilla ticket associated with the CVE.
2. **Examine the ProdSec Metadata:** Look for the CVSS vector string (e.g., `CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H`).
3. **Calculate/Verify:** Use the [NVD CVSS Calculator](<https://nvd.nist.gov/vuln-metrics/cvss>) to input the vector and confirm if the assigned severity (e.g., "Important") aligns with the calculated score.
4. **Identify the SLA:** Based on the internal ProdSec documentation, determine the "Due Date" for this specific severity level.
5. **Document Findings:** Update the Jira ticket with your findings regarding the reachability analysis (referencing the upcoming section on `govulncheck` if applicable to the component).

Scope of Sustaining Engineering in RHEL maintenance

Scope of Sustaining Engineering in RHEL maintenance

In the lifecycle of Red Hat Enterprise Linux (RHEL), Sustaining Engineering serves as the critical safety net. While core product engineering focuses on innovation and the development of new features for upcoming releases, Sustaining Engineering is dedicated to the long-term stability, security, and integrity of existing RHEL releases.

Defining the Mandate

The primary mandate of Sustaining Engineering is to provide **lifecycle stability**. When a RHEL release moves out of its initial active development phase, it enters various support streams such as:

- **EUS (Extended Update Support):** Providing extended support for specific minor releases.
- **TUS (Telco Update Support):** Addressing the rigorous requirements of telecommunications infrastructure.
- **E4S (Update Services for SAP Solutions):** Tailored maintenance for enterprise SAP environments.
- **ELS (Extended Life Phase):** Managing the final stages of a product's lifecycle where only the most critical security vulnerabilities are addressed.

The scope of this work is strictly defined by the need to maintain **API/ABI compatibility**. Unlike upstream community projects that frequently rebase to the latest version to introduce features,

Sustaining Engineering ensures that the operating system remains a predictable platform for enterprise applications.

FIXME: [Insert a block diagram here illustrating the RHEL Lifecycle phases, showing the transition from Active Development to Sustaining Engineering support streams like EUS, TUS, and ELS.]

The Backporting Strategy

One of the most defining aspects of the Sustaining Engineering scope is the **Backporting Strategy**.

When a security vulnerability is identified, the standard upstream solution is often to upgrade the software to a newer version. However, a full rebase could introduce breaking changes to APIs or shared libraries, which would force customers to re-certify their applications.

To mitigate this, Sustaining Engineering engineers:

1. **Analyze the Upstream Fix:** Isolate the specific code changes required to resolve the vulnerability.
2. **Adapt for Stability:** Modify (backport) these changes to apply cleanly to the older, stable codebase maintained in the RHEL dist-git repository.
3. **Validate Integrity:** Ensure that the patched code does not alter the package's behavior or performance profile, maintaining full binary compatibility.

Coordination and Impact

Sustaining Engineering operates at the intersection of technical maintenance and customer assurance. The scope includes:

- **Security Patching:** Addressing CVEs (Common Vulnerabilities and Exposures) with a focus on **Critical** and **Important** severity ratings.
- **Regressions Prevention:** Working closely with QE teams to ensure that patches do not introduce side effects.
- **Errata Release:** Preparing packages for release via the Red Hat CDN, ensuring that security advisories are accurate and that the fix reaches customers through the standard update channels (e.g., `dnf update`).

Hands-on Activity: Identifying Package Scope

In this activity, you will identify if a package in your local environment is currently tracked for security updates within the Sustaining Engineering scope.

1. **Clone the dist-git repository:** Use `rhpkg` to access the source for the package you are investigating.

```
rhpkg clone <package-name>
cd <package-name>
```

1. **Inspect the Spec File:** Check the `.spec` file to see the version history and past security patches.

```
grep -i "Patch" *.spec | head -n 10
```

1. **Review ChangeLogs:** Examine the changelog entries to understand the history of backported security fixes vs. major version bumps.

```
rpm -qp --changelog <package-name>.spec | grep -i "CVE"
```

Note: If you find recent entries referencing specific CVEs, you are looking at the direct output of the Sustaining Engineering maintenance process.

Strategies for backporting security patches versus upstream rebasing

Strategies for Backporting Security Patches versus Upstream Rebasing

In the lifecycle of Red Hat Enterprise Linux (RHEL), maintaining the balance between security and stability is paramount. When a vulnerability is identified, Sustaining Engineering teams must decide whether to **backport** a specific security fix or perform an **upstream rebase** of the entire package.

Understanding the Trade-offs

Choosing the correct strategy depends on the maturity of the RHEL release, the complexity of the security fix, and the potential impact on API/ABI compatibility.

Backporting Security Patches

Backporting is the practice of taking a fix from a newer version of software (upstream) and applying it to an older version (RHEL) without upgrading the entire software stack.

- **Advantages:**
- **Minimal Risk:** Only the specific vulnerable code is modified, reducing the chance of introducing regressions.
- **API/ABI Stability:** The core functionality and binary interfaces remain unchanged, which is critical for RHEL's support guarantees.
- **Focused Validation:** Testing efforts can be narrowed to the affected component and its immediate dependencies.
- **Challenges:**
- **Complexity:** As the gap between RHEL and upstream widens, applying patches often requires manual adjustments (e.g., resolving hunk failures).
- **Maintenance Burden:** Long-term maintenance of custom-patched packages can become difficult if security fixes become increasingly disconnected from upstream patterns.

Upstream Rebasing

Upstream rebasing involves updating the RHEL package to a newer version provided by the original software maintainers.

- **Advantages:**
- **Alignment:** Synchronizes the package with current upstream standards, incorporating new features and performance improvements.
- **Simplification:** Eliminates the need to manage custom patches, as the fix is part of the newer upstream codebase.
- **Challenges:**
- **Regressions:** New code often brings new bugs or changes in behavior that can disrupt existing enterprise workflows.
- **Compatibility:** Upstream updates may break API/ABI compatibility, potentially breaking dependent applications or requiring a rebuild of the ecosystem.

FIXME: [Insert flow diagram here illustrating the decision tree: 'Vulnerability Identified' → 'Evaluate Version Gap' → 'Low Gap/Stable Interface? Yes: Backport / No: Consider Rebase']

Decision Matrix for Sustaining Engineering

When determining the strategy, engineers should evaluate the following criteria:

Criteria	Consideration
Severity	Critical/Important CVEs often necessitate a fast, surgical backport to minimize exposure.
Release Stage	Early-stage RHEL versions may support rebases; ELS (Extended Life Cycle Support) streams strictly favor backporting to ensure maximum stability.
Code Delta	If the upstream fix requires refactoring a significant portion of the codebase, a rebase might be safer than a complex backport.
Upstream Health	Is the upstream release stable and well-supported? If the upstream version is experimental, backporting remains the preferred choice.

Hands-on Activity: Identifying Patch Applicability

To determine if a backport is feasible, use `rhpkg` to inspect the current state of the source code and the impact of the patch.

1. **Retrieve the source package:** Ensure you are in the dist-git directory for the target package.
[source,bash] --- `rhpkg prep` ---
2. **Verify Patch Cleanliness:** Apply the patch locally to see if it merges cleanly into the RHEL source tree. [source,bash] --- `patch -p1 < path/to/security-fix.patch` ---
3. **Analyze Hunk Failures:** If `patch` returns "hunk failed," it indicates the code structure has drifted too far from the upstream version. At this stage, you must:
 - Manually resolve the conflicts.
 - Evaluate if a rebase is required due to the architectural divergence.
4. **Lint for Quality:** Always validate your spec file changes before proceeding to build.
[source,bash] --- `rhpkg lint` ---

CVE Resolution Workflow

CVE Resolution Workflow

The CVE resolution workflow is a structured, multi-stage lifecycle designed to address security vulnerabilities within RHEL while maintaining stability through targeted backporting.

FIXME: [Block diagram representing the CVE Resolution Lifecycle: Triage → Analysis → Development → Build → Testing → Release]

The process consists of the following key stages:

- **Triage and Analysis:** Product Security (ProdSec) identifies vulnerabilities and assigns severity via CVSS scores. Subsystem teams determine reachability using static analysis tools like `govulncheck` and `callgraph` to verify if vulnerable code is executed.
- **Tracking and Coordination:** Issues are managed via Jira and Bugzilla. ProdSec tools establish SLAs and manage embargo guidelines to ensure compliance with security requirements.
- **Development:** Engineers apply minimal, targeted security patches to the relevant RHEL release stream using `rhpkg` and dist-git repositories, prioritizing stability over rebasing.
- **Build:** Validated code is submitted to the Brew build system to generate secure RPM packages.
- **Testing and QE:** Fixes undergo rigorous validation using Beaker, `tmt`, and Testing Farm to ensure the resolution is effective and does not introduce regressions.
- **Release:** The finished build is attached to the Errata Tool (RHSA). Following successful quality checks (TPS/CAT), the update is published via the Content Delivery Network (CDN).

Lifecycle stages: Triage, Analysis, Development, Build, Testing, and Release

CVE Resolution Lifecycle: From Triage to Release

The CVE (Common Vulnerabilities and Exposures) resolution process within Red Hat Enterprise Linux (RHEL) is a highly orchestrated workflow designed to ensure security, stability, and reliability. Sustaining Engineering teams follow a standardized lifecycle to move from the identification of a vulnerability to the delivery of a tested, secure update to customers.

FIXME: [Insert flowchart visualizing the CVE lifecycle: Triage → Analysis → Development → Build → Testing → Release]

Lifecycle Stages

The resolution process is divided into six distinct, interconnected stages:

1. Triage

The process begins with the **Product Security (ProdSec)** team. Once a CVE is identified, ProdSec

performs an initial assessment to determine the severity and potential impact on RHEL. * **Tracking:** A Jira issue is created to track the vulnerability throughout its lifecycle. * **Prioritization:** The severity is assigned (often using CVSS scoring), which dictates the Service Level Agreement (SLA) for the fix. * **Embargo Management:** If the vulnerability is under embargo, coordination is required to ensure information is not leaked before the official release.

2. Analysis

Before a fix is written, developers and engineers must verify the vulnerability. * **Impact Assessment:** Determine if the RHEL package is actually affected. * **Reachability Analysis:** Using tools like `govulncheck` (for Go-based components) or static analysis/callgraph tools (for C/C++), the team confirms if the vulnerable code path is actually executable within the RHEL context. This prevents unnecessary patching of code that cannot be reached by an attacker.

3. Development

Once confirmed, the team develops the fix. * **Backporting vs. Rebasing:** RHEL prioritizes stability. Developers typically **backport** specific security patches to the existing version of the package in the `dist-git` repository. Upstream rebasing is avoided unless absolutely necessary to ensure the fix does not introduce regressions. * **Implementation:** The fix is committed to `dist-git` using `rhpkg`.

4. Build

After the source code is patched, the package must be compiled into an RPM. * **Brew:** This is the primary build system. Builds are performed in Brew, which provides a clean, reproducible environment. * **Embargoed Builds:** If the CVE is under embargo, builds are sent to "nocompose" targets to prevent them from accidentally being included in public repositories before the official announcement.

5. Testing

Validation is the most critical step to ensure that the fix works without breaking existing functionality. * **Automated Testing:** Teams utilize `tmt` (Test Management Tool) and the **Testing Farm** for rapid, cloud-based execution of test suites. * **Hardware/Architecture Validation:** For kernel or hardware-specific components, **Beaker** is used to schedule tests on bare-metal systems across diverse architectures (x86_64, aarch64, ppc64le, s390x). * **Regression Suite:** CI pipelines trigger automated regression tests to verify that the security fix has not introduced side effects.

6. Release

The final stage is the delivery of the update to the customer. * **Errata Tool:** The build is attached to an Errata record, specifically a Red Hat Security Advisory (RHSA). * **Quality Gates:** The Errata must pass automated checks, including TPS (Test Planning System) and CAT (Content Approval Tool). * **Shipment:** Once finalized, the advisory is pushed to the Content Delivery Network (CDN) and the public security announcement is issued.

Hands-on Activity: Tracking a Fix

To understand the flow, follow this simulated workflow in your environment:

1. **Locate the Jira Ticket:** Open the project's Jira dashboard and identify an open CVE ticket.

2. **Verify the Build:** Use `rhpkg` to clone the source for the affected package and inspect the `dist-git` history for recent security patch commits.
3. **Run a Local Build:** Use `rhpkg build` (or your local equivalent) to trigger a test build in Brew.
4. **Schedule a Test:** Define a `plans/` directory using `tmt` within your package repo to verify the fix and run it via `tmt run`.

Coordination between Product Security and Subsystem Teams

Coordination between Product Security and Subsystem Teams

In the lifecycle of a RHEL vulnerability, effective communication between the Product Security (ProdSec) team and Subsystem Teams (SSTs) is critical to maintaining the security posture of the distribution. This coordination ensures that CVEs are not merely identified but are systematically addressed, verified, and delivered to customers within established Service Level Agreements (SLAs).

The Division of Labor

The relationship between ProdSec and SSTs is defined by a clear separation of responsibilities, allowing for both specialized security oversight and deep technical ownership.

- **Product Security (ProdSec):** Acts as the central nervous system for security vulnerabilities. They are responsible for:
 - **Triage:** Assigning CVSS scores and determining the severity (Critical, Important, Moderate, Low).
 - **Policy Enforcement:** Managing embargo guidelines and setting the strict SLA deadlines for patch delivery.
 - **Tracking:** Providing the centralized infrastructure (Jira/Bugzilla) to manage the global state of a CVE.
- **Subsystem Teams (SSTs):** Act as the technical authorities for the codebase. Their responsibilities include:
 - **Impact Analysis:** Determining if the vulnerable code is reachable within the specific RHEL component.
 - **Remediation:** Developing the technical fix, often through backporting patches to maintain ABI/API stability.
 - **Verification:** Ensuring that the fix does not introduce regressions through rigorous testing in the build pipeline.

The Coordination Workflow

Coordination is facilitated through the tracking platforms (Jira/Bugzilla). When a CVE is disclosed, ProdSec creates a "tracker" bug. The SST is then responsible for linking their internal work to this tracker. This linkage allows ProdSec to monitor the progress of fixes against the mandatory SLA.

Best Practices for Effective Coordination

To maintain velocity while ensuring security integrity, teams should adhere to the following coordination standards:

1. **Transparent Communication:** If an SLA is at risk due to technical complexity (e.g., a difficult backport), the SST must communicate the delay through the tracker bug early. This allows ProdSec to adjust expectations or provide additional resources.
2. **Embargo Adherence:** For non-public vulnerabilities, all communications and work must occur within embargoed channels to prevent premature disclosure.
3. **Cross-Functional Feedback:** SSTs should provide input to ProdSec during the impact analysis phase. If the team discovers that a vulnerability is unreachable or mitigated by other factors, the SST should provide that evidence to ProdSec to update the severity or the necessity of a fix.

Hands-On Activity: Navigating a CVE Tracker

This activity simulates the interaction between ProdSec and an SST using standard issue-tracking workflows.

Goal: Identify assigned CVEs and update the status within the tracker.

Steps: 1. **Access the Dashboard:** Log in to your organization's issue tracker (Jira/Bugzilla) and filter for bugs assigned to your component team with a "Security" label or type. 2. **Review the ProdSec Meta-Data:** Open a CVE tracker bug. Identify the assigned severity (e.g., Important) and the specific SLA due date provided by ProdSec. 3. **Perform Impact Assessment:** Using your local development environment, verify if the vulnerable function identified in the tracker is present and reachable in your component's source code. 4. **Update the Tracker:** * Change the bug status from **NEW** or **ASSIGNED** to **ON_DEV** once you begin working on the backport. * Add a comment documenting your findings regarding reachability. 5. **Submit for Peer Review:** Once the **rhpkg** build is successful, update the tracker with the build link and transition the bug status to **POST** or **MODIFIED** for QA verification.

Note: Always ensure your activities comply with current organizational embargo policies if the CVE has not yet been publicly disclosed.

Management of security advisories and errata release

Management of Security Advisories and Errata Release

In the Red Hat engineering ecosystem, the management of security advisories is the critical bridge between internal software remediation and customer-facing protection. This process ensures that verified fixes are packaged, validated, and distributed through the Content Delivery Network (CDN) in a standardized, trackable manner.

The Role of the Errata Tool (ET)

The **Errata Tool (ET)** serves as the central orchestration hub for all Red Hat Security Advisories (RHSA). It is not merely a distribution mechanism but a compliance and tracking engine that connects disparate systems:

- **Jira Integration:** Links the CVE tracking issues (containing the technical analysis) to the physical RPM builds.
- **Release Management:** Manages the lifecycle of an advisory from its initial drafting to its final publication.
- **Documentation Coordination:** Interfaces with documentation teams to ensure that the security impact and remediation steps are clearly communicated to customers.
- **QE Sign-off:** Requires formal validation from Quality Engineering (QE) before an advisory can transition to **REL_PREP** (Release Preparation).

Advisory Lifecycle Stages

When a fix is ready for release, it moves through a structured workflow to ensure that updates do not introduce regressions and meet established Service Level Agreements (SLAs).

FIXME: Add a sequence diagram illustrating the lifecycle of an Errata Advisory: Draft → QE Testing → Sign-off → REL_PREP → Push to CDN.

1. Assembly and Attachment

Once a patch has been backported to **dist-git** and built successfully in Brew, the maintainer attaches the resulting RPM builds to an RHSA in the Errata Tool. Each advisory must contain all relevant metadata, including the CVE ID, impact severity (as defined by CVSS), and the list of affected packages.

2. Validation and QE Sign-off

Before release, the advisory undergoes rigorous verification: * **TPS/CAT (Content Availability Testing):** Automated tests verify that the packages are successfully published to the CDN and can be installed cleanly on supported customer systems. * **Regression Testing:** Utilizes **tmt** (Test Management Tool) and Testing Farm to ensure that the security fix has not negatively impacted existing functionality. * **Status Updates:** Once testing is successful, the QE team provides a formal sign-off in the Errata Tool, which is a mandatory prerequisite for moving the advisory to the release phase.

3. Release Preparation and Shipment

Upon sign-off, the advisory enters **REL_PREP**. At this stage, the release engineering team schedules the push to the CDN. Once the push completes, the advisory becomes public, and customers can consume the update via standard package management tools (e.g., **dnf update**).

Hands-On Activity: Managing an Errata Advisory

In this activity, you will simulate the final stages of preparing an advisory for release.

Prerequisites: * Access to the Errata Tool (ET) staging environment. * A Jira tracking bug in **Verified** status. * A completed Brew build ID.

Steps:

1. **Link the Build:** Log into the Errata Tool and navigate to your assigned RHSA. Use the "Add Build" function to associate your **Brew** build ID with the advisory.
2. **Verify Content:** Ensure the **Packages** tab in ET correctly lists the RPMs you intend to ship.
3. **Trigger TPS:** Initiate the Content Availability Testing from the "Testing" tab to ensure the repository metadata is generated correctly.
4. **Transition to QE:** Move the advisory status from **NEW_FILES** to **QE** by updating the workflow state in ET.
5. **Final Sign-off:** Once the test results in the "Testing" tab show a "PASS" for all environments, perform the final QE sign-off.
6. **Update Jira:** Navigate to your Jira tracker and transition the bug status to **RELEASE_PENDING** to notify the Product Security team that the fix is queued for shipment.

Best Practices for Security Advisories

- **Avoid "Over-packaging":** Only include the builds necessary for the CVE resolution to minimize the risk of regressions.
- **Timely Communication:** Ensure that the RHSA documentation accurately reflects the scope of the vulnerability; unclear documentation can lead to unnecessary customer support tickets.
- **Embargo Awareness:** When handling embargoed CVEs, ensure that builds are directed to **nocompose** targets in Brew to prevent premature public availability via the CDN.

Triage and Tracking Tools

Triage and Tracking Tools

Triage and tracking tools provide the infrastructure necessary to manage the CVE lifecycle, from initial identification to final resolution. These systems ensure accountability, maintain compliance with security SLAs, and facilitate communication between Product Security (ProdSec) and Subsystem Teams (SSTs).

Key components include:

- **Issue Management (Jira/Bugzilla):** Centralized platforms used to create CVE tracker bugs, assign ownership, and track progress against defined SLAs.
- **ProdSec IR Tooling:** Specialized internal tools used to calculate CVE due dates, establish specific SLAs, and enforce embargo guidelines for sensitive vulnerabilities.
- **Severity-Driven Scoping:** Tracking requirements are determined by the vulnerability's severity; for example, Critical and Important CVEs require tracking across both Z-stream and Y-stream, whereas Moderate CVEs may only require Y-stream tracking.

FIXME: [Insert block diagram illustrating the relationship between Jira/Bugzilla, ProdSec tooling, and the workflow integration for tracking SLAs and severity-based task assignments.]

Managing issue tracking with Jira and Bugzilla

Managing Issue Tracking with Jira and Bugzilla

In the lifecycle of a CVE, effective issue tracking acts as the single source of truth. It bridges the gap between the initial discovery by Product Security (ProdSec) and the final release of an errata update. While both Jira and Bugzilla are utilized within the Red Hat ecosystem, understanding their roles and the specific workflows required for CVE management is critical for maintainers and engineers.

The Role of Jira and Bugzilla in Vulnerability Management

Bugzilla traditionally serves as the primary interface for tracking individual bugs and vulnerability reports, particularly those originating from upstream or external reporters. Jira, however, is increasingly used for project management, tracking complex workflows, and coordinating cross-team efforts during the vulnerability remediation lifecycle.

Key Tracking Requirements

Effective tracking requires adherence to specific guidelines to ensure compliance with Service Level Agreements (SLAs) and internal audit standards:

- **Severity Alignment:** Every tracker must be correctly labeled with the appropriate CVSS severity level. This determines the internal SLA for the fix.
- **Embargo Guidelines:** When a vulnerability is under embargo, issue trackers must be restricted

to authorized users. Maintainers must ensure that sensitive data, such as private Proof of Concept (PoC) scripts or early-access patches, is not exposed to non-authorized personnel.

- **State Transitions:** Trackers must follow a strict state machine (e.g., **NEW** → **ASSIGNED** → **ON_QA** → **VERIFIED**). Moving between states incorrectly can impact metrics and delay the release process.

Workflow for Issue Tracking

When a new CVE is identified, ProdSec creates an initial tracking issue. It is the responsibility of the Subsystem Team (SST) to ensure this ticket accurately reflects the progress of the remediation.

FIXME: [Insert Workflow Diagram: A flowchart showing the state transition of a CVE tracker from Triage through Development to Verified/Release Pending]

Best Practices for Issue Maintenance

To maintain high-quality tracking, follow these standardized steps:

1. **Initial Review:** Read the vulnerability description, upstream advisories, and the linked Merge Request (MR). Identify if a reproducer or PoC is attached to the tracker.
2. **Evidence Documentation:** As you work on the fix, the tracker must serve as an audit log. Post comprehensive comments that include:
 - Test execution commands (e.g., **tmt run** or **bkr** command-line strings).
 - Direct links to test run logs and results.
 - A summary of manual reproducer findings to confirm the vulnerability exists (or does not exist) in the current package version.
3. **Transitioning Status:**
 - Once a fix is developed and built, transition the tracker to **Verified** or **RELEASE_PENDING** following the Goose QE Errata validation process.
 - Ensure all related builds are linked to the tracker for traceability.

Troubleshooting Tracking Issues

If you encounter a bottleneck in the tracking process, consider the following common issues:

- **Missing SLAs:** If a tracker is missing a severity score, escalate immediately to the ProdSec team to prevent SLA breaches.
- **Visibility Issues:** Ensure that you are using the correct security level for the ticket. If you cannot see an attached PoC, verify your permissions or contact the ticket owner.
- **Synchronization Gaps:** If there is a mismatch between the state in the Errata Tool and the state in Jira, perform a manual audit to confirm that the build has been successfully tested and promoted.



Always ensure that the Errata Tool and the Jira/Bugzilla tracker are synchronized. The Errata Tool relies on the **VERIFIED** status in the issue tracker to allow the movement of an advisory to the **REL_PREP** (Release Preparation) stage.

Establishing SLAs and embargo guidelines with ProdSec tools

Establishing SLAs and Embargo Guidelines with ProdSec Tools

In the Red Hat Enterprise Linux (RHEL) ecosystem, maintaining security integrity requires strict adherence to Service Level Agreements (SLAs) and the handling of sensitive vulnerability information. This process is orchestrated through specialized Product Security (ProdSec) tooling to ensure consistent, global responses to security threats.

Understanding SLAs and Embargoes

Service Level Agreements (SLAs)

SLAs define the mandatory timeframe within which a vulnerability must be addressed. These timelines are driven by the assigned **CVSS (Common Vulnerability Scoring System)** severity. * **Critical/Important CVEs:** Demand immediate attention, often requiring full Z-stream and Y-stream trackers to ensure existing installations and future releases are patched. * **Moderate/Low CVEs:** Typically follow a more flexible schedule, often limited to Y-stream trackers depending on the specific product release policy.

Embargo Guidelines

An embargo is a period during which vulnerability information is restricted to specific parties (e.g., vendors, OS maintainers) to prevent exploitation before a fix is available. ProdSec tools are critical in: * Restricting access to Jira/Bugzilla trackers. * Ensuring communication occurs over encrypted, authorized channels. * Coordinating the "coordinated disclosure" date with upstream maintainers and other Linux distributions.

Managing Compliance via ProdSec Tooling

ProdSec internal tools automate the calculation of these deadlines, translating global security policies into actionable dates for Subsystem Teams (SSTs).

Workflow: From CVE Receipt to SLA Enforcement

FIXME: [Insert flow chart here: Showing the flow from CVE ingestion by ProdSec → Tool-based SLA calculation → Jira/Bugzilla ticket creation → SST assignment → Embargo management flags]

Step-by-Step: Utilizing ProdSec Tools for Tracking

Follow these steps to ensure compliance when handling a new CVE:

1. **Initialization:** When a new CVE is ingested, the ProdSec tool automatically calculates the remediation deadline based on the CVSS score.
2. **Tracker Setup:** A Jira or Bugzilla tracker is generated. Verify that the "Due Date" field reflects the SLA calculated by the ProdSec tool.
3. **Embargo Verification:** Check the visibility settings on the tracker. If the CVE is under embargo, the tool will restrict access to cleared personnel only.

- **Warning:** Never discuss embargoed CVEs in public channels or non-authorized Jiras.

4. **Monitoring Progress:** Use the dashboard views provided by the ProdSec suite to monitor your team's remaining time. If a blocker prevents meeting the SLA, coordinate with ProdSec immediately to request an extension or exception.

Hands-On Activity: Navigating Tracker SLAs

This activity simulates the initial triage of a vulnerability tracker.

Objective: Identify your team's deadline and visibility constraints using the provided issue tracker.

1. **Access the Tracker:** Open the Jira ticket assigned to your SST.
2. **Verify Severity:** Check the "CVSS Score" and "Severity" fields. Ensure they match the upstream advisory.
3. **Check Deadline:** Locate the "SLA Due Date" field populated by the ProdSec integration tool.
4. **Confirm Embargo Status:** Look for the security labels or "Embargoed" flags at the top of the Jira issue. If flagged, ensure your current local environment has the necessary permissions (via Kerberos/VPN) to view the issue details.
5. **Set Status:** Move the ticket to "Analysis" to signal to ProdSec that the SST has acknowledged the SLA and has begun the impact assessment.



If the calculated SLA date appears incorrect based on the vulnerability's actual impact, use the ProdSec portal to request a "Severity Re-evaluation." Do not modify the SLA date manually in Jira, as these fields are synced automatically by the backend tooling.

Tracking requirements based on vulnerability severity

Tracking Requirements Based on Vulnerability Severity

In the Red Hat Enterprise Linux (RHEL) ecosystem, vulnerability management is governed by a rigorous lifecycle defined by the severity of the identified issue. Tracking requirements are not uniform; they scale in intensity and breadth based on the Common Vulnerability Scoring System (CVSS) and the impact rating assigned by Product Security (ProdSec).

Severity-Based Tracking Tiers

The severity of a CVE determines the exposure and the required maintenance path (Y-stream vs. Z-stream). Proper tracking ensures that security patches meet the specific Service Level Agreements (SLAs) required by RHEL subscribers.

Severity	Typical Scope	Tracking Requirement
Critical / Important	Full Z-stream & Y-stream	Mandatory trackers across all active release streams (EUS, TUS, E4S, etc.).

Severity	Typical Scope	Tracking Requirement
Moderate	Y-stream only	Tracked primarily within the current active Y-stream.
Low	As needed / Backlog	Tracked at the discretion of the SST or handled in upcoming maintenance releases.

The Lifecycle of a CVE Tracker

Every vulnerability begins its lifecycle as a tracker bug in Bugzilla or a Jira issue. The metadata within these trackers is critical for:

1. **SLA Adherence:** ProdSec tools calculate strict due dates based on the assigned severity. Missing these deadlines compromises the security posture of RHEL.
2. **Embargo Management:** For vulnerabilities discovered before public disclosure, the tracker acts as the central hub for coordination, ensuring that internal communications remain restricted to authorized Subsystem Team (SST) members.
3. **Auditability:** Trackers maintain a historical record of the triage, the justification for backporting decisions, and the sign-off from QE and Security teams.

FIXME: [Insert flow chart showing the relationship between CVE Severity (Critical/Important/Moderate) and the corresponding Jira/Bugzilla tracker workflow path]

Best Practices for Tracking and Compliance

To ensure compliance with organizational standards, developers and maintainers should adhere to the following tracking requirements:

- **Initial Triage:** Upon receipt of a tracker, verify the CVSS score and severity. If the impact analysis suggests a higher risk than initially assigned, contact ProdSec immediately to adjust the tracker requirements.
- **Documentation:** Ensure that the Jira issue links all relevant upstream advisories and internal Merge Requests (MRs). This provides a clear audit trail for why specific code changes were implemented.
- **Embargo Guidelines:** If a tracker is marked as "Embargoed," ensure that no public-facing communications (e.g., public GitHub commits or mailing list discussions) occur until the CVE is officially released.
- **Release Mapping:** For Critical and Important CVEs, ensure that the build is mapped to the correct Errata Tool release. Failing to link a build to the correct tracker results in the security fix being excluded from the final advisory.

Practical Activity: Verifying Tracker Requirements

1. **Access the Tracker:** Locate your assigned CVE in the internal tracking system (Jira/Bugzilla).
2. **Review Metadata:** Identify the **Severity** field. Is it marked as **Critical** or **Important**?
3. **Verify Streams:** Check the **Fixed In** or **Affects** versions. Ensure that the tracking requirements include all relevant maintenance streams (e.g., check if the fix is tracked for both RHEL 8.x and

RHEL 9.x).

4. **SLA Verification:** Check the **Due Date** field generated by the ProdSec tool. Use this date to prioritize your development schedule against your current workload.
5. **Impact Check:** If the severity is **Important**, confirm that you have an associated reachability analysis (e.g., using **govulncheck**) attached to the ticket as part of the formal justification process.

Impact Analysis and Verification

Impact Analysis and Verification

Impact analysis and verification ensure that identified vulnerabilities are accurately scoped and effectively resolved before final release. This phase involves assessing the reachability of vulnerable code and validating the fix through rigorous testing.

- **Vulnerability Assessment:** Utilize static analysis tools to identify potential code weaknesses and determine the extent of the vulnerability within the codebase.
- **Reachability Analysis:** Employ tools like `govulncheck`, `callgraph`, and `digraph` to determine if the vulnerable code path is actually reachable during runtime, preventing unnecessary patching of non-exploitable code.
- **Verification:** Confirm the effectiveness of patches by executing manual reproducers or Proof of Concept (PoC) scripts.
- **Documentation:** Record test evidence, including command-line execution strings, logs, and summaries of findings, within the Jira/CVE tracker to ensure full transparency and compliance with QE guidelines.

FIXME: Add a flowchart illustrating the workflow from static analysis and reachability checks to final verification and documentation in Jira.

Utilizing static analysis for vulnerability assessment

Utilizing Static Analysis for Vulnerability Assessment

In the context of RHEL Sustaining Engineering, effective vulnerability management requires more than just applying patches; it requires precise **Impact Analysis**. Because RHEL prioritizes stability and binary compatibility, we must determine if a reported vulnerability is actually reachable within our specific compiled binaries before committing to a backport.

Static analysis allows us to inspect the codebase and compiled artifacts without executing them, providing a high-confidence assessment of whether a vulnerability poses a genuine threat to a specific package.

The Role of Reachability Analysis

Not every vulnerability reported in an upstream library is exploitable in a RHEL package. A vulnerability might exist in a function that is never called by our application, or the specific code path that leads to the vulnerable function might be unreachable in our build configuration.

We utilize several tools to confirm this "reachability":

- **govulncheck:** The primary tool for Go-based packages. It scans binaries and source code to identify if known vulnerable symbols are present and, more importantly, whether they are actually executed.

- **callgraph:** Used to map the relationships between functions. By generating a visual or structured representation of function calls, developers can trace if an entry point eventually triggers a vulnerable function.
- **digraph:** Often used in conjunction with callgraph to visualize the dependency graph, helping engineers understand complex linkages between different modules or libraries.

FIXME: [Insert flow chart showing the Impact Analysis workflow: Input (CVE) → Tooling (govulncheck/callgraph) → Analysis (Reachable vs Unreachable) → Decision (Patch Required vs False Positive/No Impact)]

Hands-on Activity: Assessing Reachability with Govulncheck

This activity demonstrates how to verify if a vulnerable function in a Go-based package is reachable.

Prerequisites

- Access to a RHEL environment with **govulncheck** installed.
- The source code or the compiled binary of the package under investigation.

Steps

1. **Install the tool** (if not already present):

```
sudo dnf install golang-tooling
go install golang.org/x/vuln/cmd/govulncheck@latest
```

2. **Scan the binary:** Navigate to the directory containing your compiled binary and execute:

```
govulncheck ./path/to/your/binary
```

3. **Analyze the Output:**

- If **govulncheck** reports **no vulnerabilities found**, the vulnerable functions are either not present or not called. Document this in the Jira tracker as "No Impact" or "Not Reachable."
- If vulnerabilities are found, the tool will provide a **call stack**. Review this stack to confirm if the path from **main()** to the vulnerable function is a logical execution path in your environment.

4. **Verification:** If the tool identifies a reachable path, export the call stack as evidence for the CVE tracker. If the path is unreachable, include a justification in your analysis note explaining why the code execution flow prevents the vulnerability from being triggered.

Best Practices for Technical Documentation

When documenting your static analysis findings in Bugzilla or Jira, always include:

- **Version Context:** Specify the NVR (Name-Version-Release) of the package analyzed.

- **Tool Version:** Note the version of `govulncheck` or the compiler used to generate the callgraph.
- **Evidence:** Attach the generated callgraph (in `.dot` or image format) or the `govulncheck` output logs to the tracker.
- **Clear Conclusion:** State explicitly whether the vulnerability is "Reachable" (Requires patch) or "Not Reachable" (False positive for this specific build).

Reachability analysis using govulncheck and callgraph/digraph

Reachability Analysis using govulncheck and callgraph/digraph

In the context of Red Hat Enterprise Linux (RHEL) maintenance, vulnerability assessment is not limited to identifying the presence of a vulnerable library. A critical step in the triage process is **Reachability Analysis**. This process determines whether a vulnerable function within a dependency is actually reachable—and thus exploitable—within the context of the specific component being analyzed.

By confirming if an execution path exists from the component's entry points to the vulnerable function, engineering teams can prioritize patches effectively and reduce the noise of false positives.

Understanding Reachability Analysis

Not every vulnerable function inside a shared object or library is invoked during the normal operation of a host package. If static analysis tools prove that the code path to a vulnerable function is never exercised by the component, the actual security impact may be downgraded or dismissed.

Tools for Static Analysis

- **govulncheck:** The primary tool for Go-based components. It analyzes the component's codebase to detect if it calls any vulnerable functions from its imported dependencies.
- **callgraph / digraph:** These are used for broader compiled code analysis. By generating a call graph (the map of all functions and their callers) and analyzing it with `digraph` (which handles directed graphs), engineers can visualize whether a vulnerable symbol is reachable from the component's main or exported APIs.

FIXME: Add a flowchart illustrating the Reachability Analysis process, starting from "Input: Vulnerable Dependency" through "Static Analysis (govulncheck/callgraph)" and ending with the decision tree "Reachable vs. Unreachable."

Hands-On Activity: Performing Reachability Analysis

This activity demonstrates how to verify reachability using `govulncheck` for a Go-based component.

Prerequisites

- Access to the source code repository of the RHEL component.

- **govulncheck** installed in your build environment.
- A identified CVE associated with a specific Go module dependency.

Step-by-Step Instructions

1. **Prepare the Environment:** Navigate to the root directory of the package containing the **go.mod** file. [source,bash] ---- `cd /path/to/component/source` ----
2. **Run govulncheck:** Execute **govulncheck** to scan your codebase against the vulnerability database. [source,bash] ---- `govulncheck ./...` ----
3. **Analyze the Output:**
 - If **govulncheck** identifies the vulnerability but states "[vulnerable-function] is not called", your component is likely not impacted by the CVE, despite using the vulnerable dependency.
 - If it reports that the function is called, you must review the **Call Stack** provided in the output to understand which local function is initiating the call to the vulnerable dependency.
4. **Deep Dive with Callgraph (Advanced):** If **govulncheck** is inconclusive, generate a call graph to manually trace execution. [source,bash] ---- `# Generate the call graph in text format go install golang.org/x/tools/cmd/callgraph@latest callgraph -format text ./... > callgraph.txt` ----
5. **Verify using Digraph:** Use **digraph** to determine if a path exists between your entry point (e.g., **main.main**) and the vulnerable function symbol identified in the CVE report. [source,bash] ---- `# Filter callgraph for the specific vulnerable symbol grep "vulnerable_function_name" callgraph.txt` ----

Best Practices for Impact Assessment

- **False Positives:** Always corroborate tool output with a code review. Sometimes static analysis may miss indirect calls (e.g., via reflection or interface satisfaction).
- **Documentation:** When a vulnerability is deemed "Unreachable," document your findings in the Jira/Bugzilla ticket, including the **govulncheck** output, to provide clear evidence for the Product Security team.
- **SLA Alignment:** Use reachability analysis results to adjust the priority. If a vulnerability is unreachable, it may allow for a slower, more deliberate patching cycle compared to a critical, directly exploitable vulnerability.

Verifying vulnerable function calls in compiled code

Verifying Vulnerable Function Calls in Compiled Code

Once a CVE has been identified and a potential patch is developed, confirming the fix at the source code level is only half the battle. In the context of RHEL Sustaining Engineering, we must verify that the compiled machine code no longer contains the vulnerable logic. This process ensures that the fix was not only applied correctly but also preserved through the compilation and optimization pipeline.

Understanding the Verification Challenge

When a vulnerability is located within a specific function, simply inspecting the source code is insufficient. Compilers can perform aggressive optimizations—such as inlining, loop unrolling, or dead-code elimination—which may alter the control flow or even remove the vulnerable function entirely, or conversely, create new execution paths.

To verify the compiled binary, we must bridge the gap between the source-level fix and the binary-level reality.

Workflow: From Patch to Binary Verification

The verification process involves tracing the vulnerable function calls from the source-level logic down to the assembly level within the shared object (`.so`) or executable binary.

FIXME: [Insert flow diagram: Vulnerability Verification Pipeline: Source Code → Compiler → Optimized Binary → Binary Analysis/Disassembly → Verification Result]

Step-by-Step: Binary Analysis Procedure

Follow these steps to verify that the vulnerable function call has been remediated in the compiled RPM.

1. Extract the Compiled Artifact

Before analysis, ensure you are working with the binary generated by the patched build (Brew).

```
# Extract the RPM package to retrieve the binaries
rpm2cpio <package_name>.rpm | cpio -idmv
# Locate the target library or binary
find . -name "libvulnerable.so"
```

2. Locate the Vulnerable Symbol

Use `nm` or `objdump` to inspect the symbol table. This confirms that the vulnerable function exists in the binary and identifies its address.

```
# List symbols and filter for the vulnerable function
nm -D libvulnerable.so | grep <vulnerable_function_name>
```

3. Disassemble the Control Flow

To verify the fix, we use `objdump` to examine the assembly instructions. We are looking for the removal of dangerous instructions or the addition of boundary checks (e.g., bounds checking before a buffer copy).

```
# Disassemble the function to view the assembly code
objdump -d -j .text libvulnerable.so | sed -n '/<vulnerable_function_name>:/,/^$/p'
```

- **What to look for:**
- **Inlining:** If the function is inlined, it may not appear as a distinct call. Check if the code previously surrounding the call has changed.
- **Boundary Checks:** Look for `cmp` (compare) instructions followed by conditional jumps (`jle`, `jne`) that prevent illegal memory access.
- **Safety Primitives:** Verify that functions like `strcpy` have been replaced by their safer counterparts (`strncpy`, `strncpy`) in the disassembly.

4. Trace Reachability

If the function is large, use `callgraph` or static analysis tools to verify the call path. Ensure that the vulnerable code path is now unreachable or correctly guarded.



If `govulncheck` (for Go-based components) was used during the analysis phase, compare its output before and after the patch. It provides a high-level view of whether the vulnerable function remains in the call stack.

Troubleshooting Verification

If the binary still shows evidence of the vulnerability:

1. **Check Compiler Flags:** Ensure that `_FORTIFY_SOURCE` or stack-protector flags were active during the Brew build.
2. **Verify Debug Symbols:** If you cannot see function names, ensure the `debuginfo` package for the RPM is installed.
3. **Cross-Reference with `dist-git`:** Verify that the `rhpkg` patch was correctly applied to the sources before the build triggered. If the patch did not apply cleanly, the binary will not reflect the fix.

Summary Checklist for QE

- ☐ Binary extracted from the patched Brew build.
- ☐ Symbol verification confirms the patched code is present.
- ☐ Disassembly validates that security bounds or logic gates are active.
- ☐ No regression detected in adjacent function calls.
- ☐ Results documented in the Jira ticket for the Errata sign-off.

Build and Release Infrastructure

Build and Release Infrastructure

The build and release infrastructure ensures that security fixes are securely packaged, validated, and distributed to RHEL customers. This process relies on a coordinated toolchain to maintain code integrity and stability.

FIXME: Add a flowchart illustrating the build-to-release lifecycle: `rhpkg` (development) → Brew (build) → Testing Farm/Beaker (validation) → Errata Tool (release).

Key components include:

- **Package Management:** Developers use `rhpkg` to manage `dist-git` repositories, allowing them to track source code, update `.spec` files, and push security patches.
- **Build System:** The Brew environment is used to compile RPM packages from patched sources, ensuring consistent builds across target RHEL releases.
- **Quality Assurance:** Fixes undergo rigorous validation using the Testing Farm and `tmt` (Test Management Tool), complemented by `Beaker` for hardware-level regression testing across multiple architectures (x86_64, aarch64, s390x, ppc64le).
- **Release Distribution:** Once validated, builds are attached to the Errata Tool (RHSA). After passing internal quality gates (TPS/CAT), the security updates are published to the Red Hat Content Delivery Network (CDN) for end-user consumption.

Patching dist-git repositories using rhpkg

Patching dist-git Repositories using rhpkg

In the Red Hat Enterprise Linux (RHEL) sustaining engineering workflow, `rhpkg` serves as the primary interface for managing package source code. When a vulnerability is identified, engineers must modify the downstream `dist-git` repository to include the necessary security patches while ensuring the integrity of the RPM build process.

The Role of dist-git and rhpkg

The `dist-git` repository acts as the single source of truth for a specific RHEL package version. It contains the `.spec` file, source archives, and a collection of patches applied on top of the upstream source. `rhpkg` facilitates the lifecycle of these files, allowing engineers to pull sources, prepare environments, and push updates to the build system.

FIXME: [Insert a flowchart showing the rhpkg workflow: Clone → Prep → Patch/Edit → Lint → Commit → Push]

Hands-on Activity: Patching a Package

This procedure outlines the standard steps to apply a security fix to a RHEL package using the `rhpkg` toolset.

1. Prepare the Workspace

Before applying a patch, ensure your environment is configured for the specific RHEL release branch. Clone the repository and prepare the source tree:

```
# Clone the repository for the target branch
rhpkg clone <package-name>
cd <package-name>

# Switch to the appropriate release branch if necessary
git checkout rhel-9.x

# Decompress sources and apply existing patches to ensure a clean state
rhpkg prep
```

2. Apply the Security Fix

After extracting the upstream commit or creating a backported patch, add the patch file to your `dist-git` directory and update the `.spec` file to reference it.

1. Copy your patch file (e.g., `CVE-2023-XXXX.patch`) into the local directory.
2. Edit the `.spec` file to include the new patch:

```
Patch10: CVE-2023-XXXX.patch
...
%prep
%setup -q
%patch10 -p1
```

3. Validate the Patch Application

Use `rhpkg prep` again to verify that the new patch applies without hunk failures or conflicts.

```
# Verify the patch applies cleanly
rhpkg prep

# Run linter to ensure compliance with packaging guidelines
rhpkg lint
```

4. Commit and Push Changes

Once validated, stage your changes. It is mandatory to include the Jira tracking ID and the CVE identifier in the commit message to maintain traceability within the product security ecosystem.

```
# Stage the new patch and the updated spec file
git add CVE-2023-XXXX.patch *.spec
```

```
# Commit with required metadata
rhpkg commit -p -m "Resolves: RHEL-XXXXXX (<CVE-YYYY-XXXX>)"

# Push the changes to the remote branch
rhpkg push
```

Troubleshooting Common Issues

- **Hunk Failures:** If `rhpkg prep` fails, the upstream patch likely relies on code structures not present in the older RHEL codebase. Use `git diff` to manually adapt the patch to the downstream source.
- **Spec File Syntax:** Use `rhpkg lint` to catch common errors such as missing patch declarations or incorrect directory permissions before attempting a build in the Brew environment.
- **Branch Mismatch:** Always verify you are on the correct `rhel-x.y` branch before pushing; pushing to the wrong branch can cause instability in downstream release streams.

Building RPMs within the Brew environment

Building RPMs within the Brew Environment

In the Red Hat ecosystem, **Brew** (the internal implementation of the Koji build system) serves as the centralized infrastructure for compiling source code into installable RPM packages and container images. For security engineers and developers, understanding how to interact with Brew is critical for moving a vulnerability fix from a patched source code state to a distributable update.

The Role of Brew in CVE Resolution

When a patch is applied to a `dist-git` repository, the source code must be compiled into binary RPMs. Brew ensures that these binaries are created in a controlled, reproducible environment that matches the target distribution release (e.g., RHEL 8, RHEL 9).

For CVE remediation, Brew provides specific functionality to handle security updates, particularly those under embargo:

- **Standard Builds:** Used for routine updates where the package is immediately ready for distribution via the Content Delivery Network (CDN).
- **Scratch Builds:** Temporary builds used for validation. They do not persist in the repository and are not intended for production release.
- **Nocompose Builds:** Specialized builds used for embargoed CVEs. These allow engineers to generate signed RPMs without triggering the automated repository composition (nightly composes), ensuring the fix remains private until the public disclosure date.

FIXME: [Insert Flowchart: Lifecycle of a CVE Build showing the path from `rhpkg push` → Brew Build → Nocompose Target → Repository Composition]

Hands-on Activity: Building and Verifying RPMs

This activity covers the process of verifying your patch via a scratch build and triggering an official build for a CVE fix.

Prerequisites

- An active `rhpkg` configuration authenticated with your credentials.
- A patched `dist-git` repository.

Step 1: Performing a Scratch Build

Before committing your changes to the official branch, perform a scratch build to verify that your code compiles across all required architectures. This prevents "broken builds" in the main line.

```
# Build the source RPM and submit to Brew for scratch analysis
rhpkg scratch-build --srpm

# Alternatively, specify architectures for broader validation
rhpkg scratch-build --arches x86_64,aarch64,ppc64le,s390x
```

Step 2: Triggering an Official Build

Once your changes are committed and pushed to the relevant branch, you must trigger an official build. If you are handling an embargoed CVE, you must use a `nocompose` target to maintain confidentiality.

```
# Trigger a standard official build
rhpkg build

# Trigger an official build using a nocompose target for embargoed CVEs
# Replace <target-release> with your specific RHEL version/target
rhpkg build --target <target-release>-nocompose
```

Troubleshooting and Best Practices

- **Build Failures:** If a build fails in Brew, examine the build logs provided in the Brew web interface. Common issues include missing build dependencies (BuildRequires in the spec file) or architecture-specific compilation errors.
- **Tracking NVR:** Always record the **NVR** (Name-Version-Release) generated by Brew. You will need this NVR to link the build to an Errata Tool advisory in the subsequent phase of the workflow.
- **Dependency Management:** If your CVE fix introduces new dependencies, ensure these are available in the buildroot. Attempting to build against unavailable dependencies will result in a build failure.

- **Architecture Parity:** Always build for all supported architectures unless explicitly instructed otherwise. A fix that compiles on `x86_64` but fails on `aarch64` will prevent the update from shipping for the entire platform.

Validating fixes through Beaker, tmt, and Testing Farm

Validating Fixes through Beaker, tmt, and Testing Farm

Once a security fix has been developed and built within the Brew environment, the validation phase is critical to ensure that the patch successfully remediates the vulnerability without introducing regressions or stability issues across supported architectures.

In the RHEL ecosystem, validation is orchestrated through a tiered approach involving `tmt` (Test Management Tool), the Testing Farm, and Beaker for hardware-specific scenarios.

The Validation Lifecycle

The transition from a built RPM to a verified update follows a structured pipeline. Before official release, every fix must pass a series of automated gates.

FIXME: [Insert flow chart here: Showing the transition from Brew Build → tmt/Testing Farm (Virtualization) → Beaker (Bare Metal) → Results/Feedback Loop]

Test Management with tmt

The **Test Management Tool (tmt)** acts as the standard interface for defining and executing test plans. It allows maintainers to specify test requirements in a declarative format (usually via a `plans.fmf` file in the package repository).

- **Standardization:** `tmt` abstracts the underlying infrastructure, allowing developers to run the same test plan locally, in a CI pipeline, or within the cloud-based Testing Farm.
- **Integration:** It supports various test types, including functional tests, performance benchmarks, and regression suites.

Executing Tests via Testing Farm

The **Testing Farm** provides the on-demand, scalable infrastructure required to execute `tmt` test plans against specific RHEL builds.

1. **Request Submission:** After a build is promoted to the testing repository, a request is sent to the Testing Farm.
2. **Environment Provisioning:** The farm provisions a guest VM based on the target RHEL version and architecture.
3. **Execution:** The test plans defined in `tmt` are cloned and executed within the provisioned environment.
4. **Reporting:** Logs and artifacts are collected and mapped back to the tracker (e.g., Jira or

Bugzilla) for maintainer review.

Hardware & Multi-Architecture Validation (Beaker)

While **tmt** and Testing Farm handle the majority of userspace testing, certain packages—such as the **kernel**, device drivers, or low-level libraries—require access to specific hardware or multi-architecture bare-metal environments. This is where **Beaker** is utilized.

- **Bare-Metal Provisioning:** Beaker allows for the scheduling of tests on physical machines, which is essential for validating kernel-mode vulnerabilities that cannot be reproduced or verified in a virtualized container.
- **Architecture Coverage:** It ensures that fixes are validated across the full RHEL support matrix (e.g., x86_64, aarch64, ppc64le, s390x).
- **Hardware Labs:** Beaker manages the physical lab infrastructure, allowing maintainers to request specific hardware configurations (e.g., specific CPU features, NICs, or storage controllers) required to verify a fix.

Hands-on Activity: Validating a Fix

This activity demonstrates how to trigger an automated test suite for a package fix.

1. **Step 1: Define the test plan** Ensure your package directory contains a **plans/** directory with a **plans.fmf** file defining the scope of your verification (e.g., testing the specific CVE reproducer).
2. **Step 2: Submit to Testing Farm** Use the **tmt** CLI or the Testing Farm API to submit your build for testing:

```
# Example: Running a tmt plan against a build
tmt run --environment BUILD_ID=1234567 --plan /plans/security_regression
```

3. **Step 3: Monitor Beaker (If Applicable)** If your fix is kernel-related, use the **bkr** CLI to check the status of your hardware job:

```
# Check job status in Beaker
bkr job-status <JOB_ID>
```

4. **Step 4: Analyze Results** Review the logs provided by the Testing Farm. If the test fails, investigate the **tmt** logs to determine if the regression was caused by the patch or an environmental factor. Once tests pass, the fix is ready to proceed to the Errata Tool.

Expert Insight: Troubleshooting Validation Failures

- **Reproducer Mismatch:** If your test passes in a VM but fails on hardware, verify if your patch relies on specific CPU flags or hardware state that is only present in the Beaker environment.
- **Triage Prioritization:** If test resources are constrained, prioritize the execution of "Tier 1" regression tests to ensure core system functionality is intact before running exhaustive security-specific suites.

- **Artifact Retention:** Always ensure that `tmt` is configured to export journal logs and core dumps, as these are indispensable when a fix fails during automated testing.

Shipping updates via the Errata Tool and CDN

Shipping Updates via the Errata Tool and CDN

Once a security fix has been developed, built in Brew, and verified through automated testing (tmt/Testing Farm) and QE sign-off, the final phase of the CVE lifecycle is the release process. This ensures that the validated security patches reach customers securely and reliably via the Content Delivery Network (CDN).

The Role of the Errata Tool (ET)

The Errata Tool serves as the central orchestration platform for releasing Red Hat Security Advisories (RHSA). It bridges the gap between the internal development process and the external customer ecosystem.

Key responsibilities of the Errata Tool in the release phase include: * **Advisory Assembly:** Linking verified Brew builds (RPMs) to the original CVE Jira tracker. * **Metadata Management:** Managing the security impact, release notes, and advisory documentation. * **Sign-off Orchestration:** Coordinating necessary approvals from Product Security, QE, and Engineering. * **Release Scheduling:** Managing the "push" process to internal and external distribution repositories.

Content Availability and Verification

Before an advisory is marked for public release, it must undergo **Content Availability Testing (CAT)** and **Test Product Systems (TPS)**. These automated tests are triggered within the Errata Tool to ensure that:

1. **Repository Sync:** The RPMs are correctly placed in the designated CDN repositories.
2. **Metadata Integrity:** The `repomd.xml` and package metadata are consistent and parseable by package managers like `dnf` or `yum`.
3. **Installation Validation:** Automated agents attempt to install the newly updated packages to confirm no dependency conflicts or installation failures exist.

FIXME: [Insert flow diagram here illustrating the transition from "QE Sign-off" to "CDN Distribution" via the Errata Tool and TPS/CAT verification]

Workflow: Moving from QE to Release

To successfully ship an update, the following workflow is performed within the Errata Tool:

1. **Transitioning Status:** Once QE confirms that the fix satisfies the security requirements and regression tests, the Errata is moved to `REL_PREP` (Release Preparation).
2. **Final Verification:** Automated TPS/CAT checks are initiated. You must monitor these statuses in the Errata Tool to ensure no "Fail" flags are present.
3. **Shipment:** Upon successful completion of automated tests and final approval, the Errata Tool

triggers the synchronization of packages to the production CDN.

Hands-On Activity: Validating Errata Readiness

In this activity, you will verify the state of an advisory before its public release.

Prerequisites: * Access to the Errata Tool instance. * An active Errata ID linked to your Jira tracker.

Steps:

1. **Review Build Attachments:** Navigate to your Errata in the Errata Tool. Ensure the correct Brew build NVRs (Name-Version-Release) are attached under the "Builds" tab.
2. **Check Automated Tests:** Look at the "Test" or "Automation" tab. Verify that the TPS and CAT status indicators are green (Pass). **If status is 'Failed', inspect the logs provided in the ET link to determine if the issue is a infrastructure flake or a genuine package dependency conflict.**
3. **Verify Documentation:** Ensure the "Advisory" tab contains the correct CVE identifier and the summary of the vulnerability.
4. **Request/Approve Sign-off:** If you have the necessary permissions, toggle the QE sign-off status to **Verified**. This signals to the release engineering team that the advisory is ready for the final CDN push.
5. **Transition Status:** Move the Errata state to **REL_PREP** to begin the final publication window.

Troubleshooting Tips

- **TPS Failures:** If TPS fails, it often indicates a missing dependency in the build or an issue with the repository composition. Always check if the required **nocompose** flags were cleared after the embargo lifted.
- **CDN Lag:** Occasionally, there is a short propagation delay between the Errata Tool status change and the appearance of packages on public mirrors. Use the **repomd** check provided in the ET interface to verify the latest metadata timestamp.

Appendix

FIXME: Content for Appendix...

Resources

[Download Course PDF](#)